

Introduction to Game Theory - Coursework I

Optimising Aim and Endgame Strategy in Darts

Authors:

Adam Marshall (CID 02411960)

Zachary Hemphill (CID 02389055)

Leo Rogers (CID 02391392)

Gabe Gourlay (CID 02382101)

November 2025

1 Introduction

Darts is conventionally viewed as a game of physical dexterity with the objective being simply to hit a target with high precision. However, from a game theory perspective, it represents a complex problem of optimisation under uncertainty. A rational player must choose a target coordinate $\mu = (x, y)$ to maximise their expected payoff. This is subject to the inherent variance in their throwing accuracy.

In a standard game of 501, the board offers various scoring regions, with the Triple 20 (T20) yielding the highest maximum score of 60. Intuitively, this suggests that aiming for T20 is the optimal strategy. Indeed, for professional players with higher accuracy, this holds true. However, for amateur players with lower accuracy, aiming for the T20 carries a significant risk of hitting adjacent low-scoring zones, such as the 1 or 5 segments.

In this paper, we analyse the game of Darts using two distinct game theory frameworks, one based on the early game (trying to get to a finishing position of less than or equal to 170), followed by the optimal strategy for an end game position, derived from the course material:

- **Strategic Dominance under Uncertainty (Optimisation):** We model the scoring phase as a single-player game throwing one dart, as this can easily be applied to each dart throw from the player where the solo aim is to maximise score. By applying the concept of Expected Payoff (Chapter 3) to a continuous strategy set, we demonstrate that for players above a certain variance threshold σ , the strategy “Aim for T20” is strictly dominated by aiming for the Triple 19 or the bottom-left quadrant of the board.
- **Zero-Sum Endgame Analysis:** In the later stages of darts, a player must finish on a specific double to make sure they end on exactly 0. So, the game shifts to a strictly competitive interaction. We model this as a Zero-Sum Game (Chapter 4), where the utility of winning is +1 and losing is -1. We derive the indifference points that determine when a rational player should attempt a risky “Aggressive” shot (e.g., Bullseye) versus a “Safe” setup shot, depending on the opponent’s likelihood of winning in the next turn.

By quantifying skill as a standard deviation σ , we derive a result showing that the optimal strategy varies. We aim to show that rationality in Darts requires a player to adjust their strategy based on a realistic assessment of their own ability.

The remainder of this paper is organised as follows: Section 2 formalises the game theoretic definitions and physical assumptions of the model. Section 3 details the computational methodology used to overcome the analytic intractability of the scoring function. Then presents the optimisation results, demonstrating the shift in optimal strategies across skill levels. Section 4 extends the analysis to the zero-sum endgame, deriving the critical indifference points for strategic aggression. Finally, Section 5 discusses the limitations of the circular variance assumption.

2 The Mathematical Model

To apply game theory analysis to Darts, we must formalise the physical mechanics of the throw as a probabilistic event. We model the player not as a deterministic agent, but as a stochastic actor whose “strategy” is the choice of a target mean μ , and whose “payoff” is the expected value of the resulting score distribution.

2.1 Game Theoretic Formulation

Formally, we define the game of Darts as a tuple (N, S, u) :

- **Players (N):** In the scoring phase, $N = 1$. In the Zero-Sum Game, $N = 2$ (Player A vs. Player B).
- **Strategy Set (S):** The set of pure strategies is continuous and infinite. A strategy $s \in S$ corresponds to a chosen aim point $\mu = (x, y)$ on the Cartesian plane of the board $\mathbb{R}^2$.
- **Payoff Function (u):** The utility function is risk-neutral. For the scoring phase, $u(\mu) = E[\text{Score}]$. For the Zero-Sum Game, $u(\mu) \in \{+1, -1\}$.

2.2 Assumptions

Following standard game theory modelling practices, we make the following simplifying assumptions to render the problem tractable:

- **Circular Variance:** We assume the player’s error is equal in all directions. Although many studies suggest vertical error is often greater than horizontal error. We model using Bivariate Normal Distribution with $\sigma_x = \sigma_y = \sigma$ to provide a robust approximation for strategic analysis.
- **Independence of Throws:** We assume each throw is an independent and identically distributed (i.i.d.) event. The result of a previous dart does not physically alter the probability distribution of the next dart (ignoring psychological factors or “marker” darts).
- **Static Skill Level:** The player’s variance parameter σ remains constant throughout the simulation. We do not model fatigue or any other factors that vary skill.
- **Single-Throw Optimisation:** A standard turn in Darts consists of three throws. To maintain mathematical tractability, we model the decision process as a per-dart optimisation problem. We assume the player re-evaluates their optimal target μ before every individual throw, independent of the remaining darts in hand, unless specifically analysing a multi-dart checkout sequence.
- **Sequential Perfect Information:** We assume the game is played sequentially with perfect information. Both players are fully aware of the current scores and the opponent’s skill level σ at all times. There is no hidden information or simultaneous movement.
- **Risk Neutrality:** We assume the players are rational and seek to maximise their Expected Payoff (score or win probability) linearly. We disregard psychological factors such as “pressure” or the utility of “glory”.

2.3 The Scoring Function $S(x, y)$

The dartboard is defined as a bounded 2D plane. We define a discontinuous scoring function $S(x, y)$ that maps any landing coordinate (x, y) to a discrete integer score $v \in \{0, 1, \dots, 60\}$.

The board is constructed of concentric rings and sector angles. Converting to polar coordinates (r, θ) for ease of definition:

$$S(r, \theta) = \begin{cases} 50 & \text{if } r < r_{bull} \\ 25 & \text{if } r_{bull} \leq r < r_{outer} \\ M(\theta) \cdot R(r) & \text{if } r \geq r_{outer} \end{cases} \quad (1)$$

where $M(\theta)$ is the base multiplier determined by the angle (the 20-slice, the 1-slice, etc.), and $R(r)$ is the ring multiplier (Treble $\times 3$, Double $\times 2$, Single $\times 1$).

We parameterise the board geometry using standard PDC (Professional Darts Corporation) measurements:

- Bullseye radius: $r_{bull} = 6.35$ mm
- Outer Bull radius: $r_{outer} = 15.9$ mm
- Treble Ring: $99 \text{ mm} \leq r \leq 109 \text{ mm}$
- Double Ring: $162 \text{ mm} \leq r \leq 170 \text{ mm}$

The angular width of each scoring segment is $\theta = 18^\circ$ ($\pi/10$ radians).

2.4 Modeling Skill Variance

We define a player’s strategy as the choice of an aim point $\mu = (\mu_x, \mu_y)$. However, due to physiological noise, the actual landing position (X, Y) is a random variable following a Bivariate Normal Distribution centered at μ .

The Probability Density Function (PDF), denoted as $f(x, y|\mu, \sigma)$, is given by:

$$f(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{(x - \mu_x)^2 + (y - \mu_y)^2}{2\sigma^2}\right) \quad (2)$$

Here, σ (standard deviation) serves as our proxy for “Skill”. A lower σ indicates high precision (Professional), while a higher σ indicates low precision (Amateur).

2.5 The Objective Function: Expected Payoff

In accordance with the definition of Expected Payoff for continuous strategy sets, a rational player seeks to choose the vector μ that maximises the expected score $E[\text{Score}]$. This is defined by the double integral of the scoring function weighted by the probability density function over the surface of the board $\mathcal{B}$:

$$\max_{\mu} E[\text{Score}] = \max_{\mu} \iint_{\mathcal{B}} S(x, y) \cdot f(x, y|\mu, \sigma) dx dy \quad (3)$$

Since $S(x, y)$ is non-continuous as it containing step-changes at the wire boundaries, this integral has no simple closed-form analytic solution. Therefore, in Section 3, we employ a discrete numerical integration method to approximate the expected value efficiently across the entire domain.

3 Model Implementation

To determine the optimal strategy for a given skill level σ , we implemented a computational simulation in Python using the NumPy library. As noted in Equation (3), the analytic integration of the expected value is intractable. We therefore approximate the continuous integral via discrete summation over a high-resolution grid.

3.1 Discretisation of the Domain

We represent the dartboard as a discrete grid G of size $N \times N$. We selected a spatial domain of 360×360 mm centered at $(0, 0)$, encompassing the entire scoring area. The grid resolution Δ was set to 1 mm, resulting in a matrix of coordinates (x_i, y_j) .

We first construct a "Perfect Aim" scoring matrix $\mathbf{S}$, where each element S_{ij} corresponds to the score a player would achieve if they hit the coordinate (x_i, y_j) with zero variance:

$$\mathbf{S}_{ij} = S(x_i, y_j) \quad (4)$$

This matrix encodes the geometry of the board as defined in Section 2.3.

3.2 Numerical Integration

We define the player's error profile as a probability matrix $\mathbf{P}$. For a given skill σ , this matrix represents the probability of a dart landing at any offset (dx, dy) from the aim point, derived from the PDF in Equation (2).

To calculate the Expected Payoff matrix $\mathbf{E}$, we compute the weighted sum of the scoring matrix $\mathbf{S}$ against the probability matrix $\mathbf{P}$ for every possible aim point. Mathematically, this discrete approximation is given by:

$$\mathbf{E}_{xy} \approx \sum_i \sum_j \mathbf{S}_{ij} \cdot P(\text{landing at } i, j | \text{aiming at } x, y) \quad (5)$$

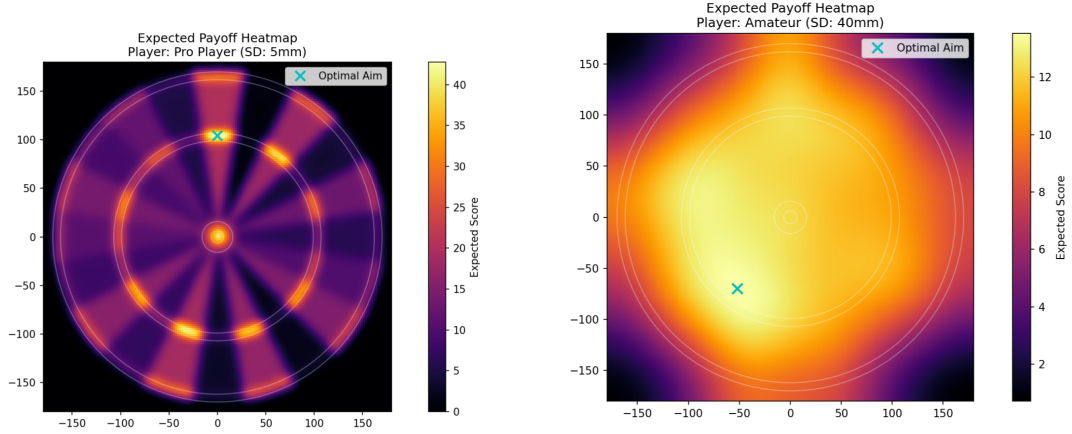
This operation (mathematically equivalent to a discrete convolution) allows us to efficiently evaluate the expected payoff for the entire board simultaneously.

3.3 Heatmap Generation and Optimisation

The resulting matrix $\mathbf{E}$ represents a scalar field where the value at each index (i, j) is the expected average score if the player aims at (x_i, y_j) . To find the optimal strategy μ^* , we simply locate the coordinate indices that maximise this matrix:

$$\mu^* = \arg \max_{(x, y)} \mathbf{E}(x, y) \quad (6)$$

We visualised these results as heatmaps. Figure 1 illustrates the contrast in optimal strategy between a professional-level player and an amateur.



(a) High Skill ($\sigma = 5\text{mm}$). The optimal strategy is strictly T20.

(b) Low Skill ($\sigma = 40\text{mm}$). The optimal strategy shifts to T19.

Figure 1: Comparison of Expected Payoff Heatmaps. (a) Shows the intuitive T20 strategy for pros. (b) Shows the strategic shift for amateurs.

For the low-variance player ($\sigma = 5\text{mm}$), the heat concentration is tight around the Treble 20, confirming intuitive strategy. However, for the high-variance player ($\sigma = 40\text{mm}$), the T20 becomes a sub-optimal spot due to the high probability of hitting the adjacent 1 and 5 sectors. The heatmap reveals that the optimal strategy shifts towards the bottom-left quadrant (near the 19, 7, and 16), where the penalty for missing the primary target is statistically lower.

3.4 Strategic Implications for the Scoring Phase

These results define the optimal policy for the *Scoring Phase* of the game (typically when the remaining score $S > 170$).

In this phase the game is not yet strictly competitive in a zero-sum sense. It can be better considered as a race. The player's objective is to minimise the expected number of darts required to reach a point from which they can checkout. Mathematically, this is equivalent to maximising the expected score per throw.

Therefore, the optimal strategy μ^* derived from our numerical integration is a **dominant strategy** during this phase. It is independent of the opponent's score. Once we enter the checkout range, the assumption of risk neutrality ($u(\mu) = E[\text{Score}]$) collapses. The utility function becomes a binary Win/Loss, and the opponent's state becomes a critical consideration. We explore this strategic shift in the following section.

4 Zero-Sum Endgame Analysis

Having established the optimal physical targeting policy (μ^*) for score accumulation in Section 3, we now address the critical 'phase transition' that occurs when a player reaches a checkout score. In this final phase, the game's nature fundamentally shifts. The objective is no longer to maximise the expected value of a distribution (Optimization), but to maximise the probability

of finishing before the opponent (Strategic Interaction). Consequently, we must abandon the single-player framework and model the endgame as a strictly competitive Zero-Sum Game.

We model this endgame phase as a **Zero-Sum Game**, as discussed in Chapter 4 of the course notes.

4.1 Game Formulation

We define the game as a tuple (N, S, u) where:

- **Players (N):** $N = \{A, B\}$.
- **Strategy Set (S):** The continuous set of aiming coordinates $\mu \in \mathbb{R}^2$, consistent with Section 2.
- **Utility (u):** $u \in \{+1, -1\}$. A win yields $+1$, a loss yields -1 .

State Dependence: The utility function u is conditional on the score vector $k = (k_A, k_B)$. We analyse the specific subgame defined by the state $k_A = 50$.

As the game is strictly competitive, we seek to maximise the *Value of the Game* (v). Note, maximising the Expected Utility $E[u]$ is mathematically equivalent to maximising the Probability of Winning (P_{win}):

$$E[u] = (+1)P_{\text{win}} + (-1)(1 - P_{\text{win}}) = 2P_{\text{win}} - 1 \quad (7)$$

Therefore, finding the Subgame Perfect Equilibrium (SPE) is the same as maximising P_{win} .

4.2 The 50 Checkout Scenario

We take a specific endgame scenario to analyse which will demonstrate the strategic shift. Player A is on a score of 50 with one dart remaining. We restrict our analysis to the final dart to isolate the decision into a binary choice:

1. **Strategy μ_{Agg} (Aggressive):** Attempt to checkout immediately by aiming for the Bulls-eye (50). This guarantees an immediate win if successful.
2. **Strategy μ_{Safe} (Safe):** Aim for a large single segment (e.g., 10 or 18) to leave a manageable double (e.g., Double 20) for the next turn. This yields a high probability of a good setup but voluntarily yields the initiative to Player B.

4.3 Determining Critical Parameters

We define the following probabilities based on Player A's skill level (standard deviation σ) and the game state:

- p_{bull} : The probability Player A hits the Bullseye given σ .
- p_{next} : The probability Player A checks out on their *next* turn, assuming a successful setup shot.
- q_{opp} : The probability Player B checks out on their ensuing turn (The "Opponent Threat").

We can define these values using our code from earlier, which can be found in the appendix. For a professional player, modelled with $\sigma = 10$ mm, $p_{\text{bull}} = 0.183$, and $p_{\text{next}} = p_{\text{setup}} * p_{\text{double}} = 0.963 * 0.315 = 0.303$.

4.4 The Indifference Equation

We calculate the Probability of Winning (P_{win}) for both strategies via backward induction.

Case 1: Aggressive Play (μ_{Agg})

Player A wins immediately if they hit the Bullseye (p_{bull}). If they miss ($1 - p_{\text{bull}}$), it becomes their opponents turn. They then only win if B misses ($1 - q_{\text{opp}}$) and A hits on the subsequent turn with a reduced probability p_{err} (this is because missing the Bull often leaves a difficult number to close from).

$$P_{\text{win}}(\mu_{\text{Agg}}) = p_{\text{bull}} + (1 - p_{\text{bull}})(1 - q_{\text{opp}})p_{\text{err}} \quad (8)$$

Case 2: Safe Play (μ_{Safe})

Player A opts for a safer approach this turn by setting up an easier future shot. They rely entirely on Player B missing ($1 - q_{\text{opp}}$) and hitting their setup shot on the next turn (p_{next}).

$$P_{\text{win}}(\mu_{\text{Safe}}) = (1 - q_{\text{opp}})p_{\text{next}} \quad (9)$$

The Indifference Point

To be rational a player aims for the Bullseye if

$$P_{\text{win}}(\mu_{\text{Agg}}) > P_{\text{win}}(\mu_{\text{Safe}}). \quad (10)$$

To derive the critical threshold for the opponent's threat level (q_{critical}), we consider the lower bound of aggression. We assume that missing the bull is a fatal mistake to the player's chances relative to the setup strategy (i.e., $p_{\text{err}} \approx 0$). This is justified because missing the Bull leaves a difficult, low probability residual score, while the Safe strategy guarantees a high probability checkout attempt. Substituting into (10):

$$p_{\text{bull}} > (1 - q_{\text{opp}})p_{\text{next}} \quad (11)$$

Solving for q_{opp} :

$$q_{\text{opp}} > 1 - \frac{p_{\text{bull}}}{p_{\text{next}}} \quad (12)$$

4.5 Results and Discussion

The inequality (12) yields a key insight into Zero-Sum dynamics. Using our previously calculated values for a pro player ($p_{\text{bull}} = 0.183$, $p_{\text{next}} = 0.303$). The critical threshold is:

$$q_{\text{critical}} = 1 - \frac{0.183}{0.303} = 0.396 \quad (13)$$

This implies that if the opponent has a greater than 39.6% chance of checking out, Player A **must** aim for the Bullseye, despite having only an 18.3% success rate.

Figure 2 shows the point where the optimal strategy switches. Where the opponent's threat (q_{opp}) exceeds the Indifference Point (≈ 0.39) and the aggressive strategy becomes superior to the safe strategy. This result matches the Minimax criterion, a player's rational choice is to maximise their minimum guaranteed security level against the threat posed by q_{opp} . We observe the safe strategy is more sensitive to the opponent's performance than being aggressive. When q_{opp} is high, the security level of the safe strategy collapses. The risky strategy μ_{Agg} which offers a strictly positive variance independent of the opponent's immediate next throw, dominates the safe strategy.

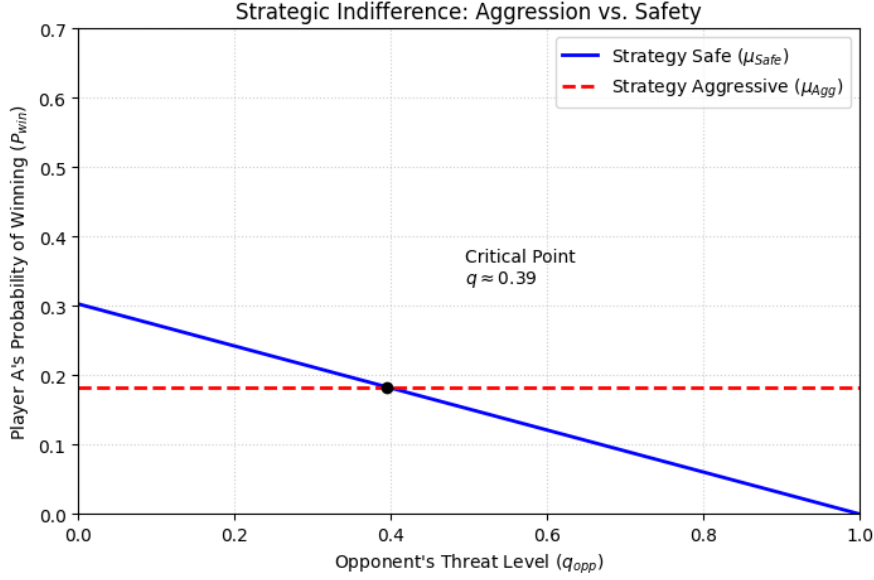


Figure 2: Zero-Sum Endgame Analysis: Strategic Indifference Point.

5 Conclusion and Limitations

This analysis reveals that the game of Darts is a hybrid mix of phases requiring distinct strategic frameworks depending on the phase of play. In the **scoring phase**, the objective is maximising our Expected Payoff ($E[\text{Score}]$). Our results demonstrate that for amateur players (high σ), the intuitive strategy of aiming for Triple 20 is **strictly dominated** and rationality demands aiming for the Triple 19 or the bottom left quadrant to mitigate the greater missing penalties. Conversely, the **checkout phase** transforms the game into a strict **Zero-Sum** competition where strategies are conditional on the opponent's threat level. For an example game state we established the **Indifference Point** ($q_{\text{critical}} \approx 0.39$) marking where the aggressive, low probability shot (Bullseye) become the optimal **Minimax criterion** strategy. When the opponent's threat level (q_{opp}) is high, this choice maximises the security against losing. However, these results are derived under key simplifications: the model ignores biomechanical realities of Elliptical Variance where $\sigma_y > \sigma_x$ (our field of vision leads to greater variance vertically than horizontally), the Hot Hand Fallacy which assumes static skill σ despite changing psychological pressures, and the Robin Hood Problem, this is the physical nature of the dart ($\approx 8\text{mm}$ width) requiring complex re-optimization if one dart is blocking the target area. Addressing these limitations in future models would yield a more comprehensive, dynamically realistic solution for the rational darts player.

A Simulation and Game Theory Code

The following Python script was used to simulate dart throws, calculate hit probabilities, and model the strategic indifference curves between safe and aggressive play styles.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.patches as patches
4 from scipy.ndimage import gaussian_filter
5 from scipy.stats import multivariate_normal
6
7 class Dartboard:
8     """
9     Represents a standard dartboard geometry and scoring rules.
10    Dimensions are based on standard tournament specifications (mm).
11    """
12    def __init__(self):
13        # Standard radii in millimeters
14        self.R_BULLSEYE = 6.35      # Inner Bull (50)
15        self.R_OUTER_BULL = 15.9    # Outer Bull (25)
16        self.R_TREBLE_IN = 99.0     # Inside of Treble ring
17        self.R_TREBLE_OUT = 107.0   # Outside of Treble ring
18        self.R_DOUBLE_IN = 162.0    # Inside of Double ring
19        self.R_DOUBLE_OUT = 170.0   # Outside of Double ring (Board Edge)
20
21        # Order of numbers starting from East (0 degrees) and going Counter-
22        # Clockwise
23        self.SECTORS = [6, 13, 4, 18, 1, 20, 5, 12, 9, 14, 11, 8, 16, 7, 19, 3,
24        17, 2, 15, 10]
25
26    def get_score(self, x, y):
27        """
28        Calculate score for a single coordinate (x, y) or arrays of coordinates.
29        """
30        r = np.sqrt(x**2 + y**2)
31        theta = np.degrees(np.arctan2(y, x))
32
33        theta = np.where(theta < 0, theta + 360, theta)
34
35        is_bullseye = r < self.R_BULLSEYE
36        is_outer_bull = (r >= self.R_BULLSEYE) & (r < self.R_OUTER_BULL)
37
38        if np.isscalar(r):
39            if is_bullseye: return 50
40            if is_outer_bull: return 25
41            if r > self.R_DOUBLE_OUT: return 0
42
43        # Determine Base Sector Score
44        sector_index = ((theta + 9) // 18).astype(int) % 20
45        base_scores = np.array(self.SECTORS)[sector_index]
46        multiplier = np.ones_like(r)
47
48        in_treble = (r >= self.R_TREBLE_IN) & (r <= self.R_TREBLE_OUT)
49        multiplier[in_treble] = 3
50        in_double = (r >= self.R_DOUBLE_IN) & (r <= self.R_DOUBLE_OUT)
51        multiplier[in_double] = 2
52        final_score = base_scores * multiplier
53
54        if not np.isscalar(r):
55            final_score[is_bullseye] = 50
56            final_score[is_outer_bull] = 25
57            final_score[r > self.R_DOUBLE_OUT] = 0
```

```

56         return final_score
57
58
59 class Player:
60     """
61     Represents a player with a specific skill level (sigma in mm).
62     """
63     def __init__(self, name, skill_sigma):
64         self.name = name
65         self.sigma = skill_sigma
66
67     def throw_darts(self, aim_x, aim_y, n_throws=1):
68         dev_x = np.random.normal(0, self.sigma, n_throws)
69         dev_y = np.random.normal(0, self.sigma, n_throws)
70         return aim_x + dev_x, aim_y + dev_y
71
72 class Simulation:
73     def __init__(self, dartboard):
74         self.board = dartboard
75
76     def visualize_heatmap(self, player, resolution=2):
77         print(f"Generating strategy heatmap for {player.name}...")
78
79         extent = 180
80         xs = np.arange(-extent, extent, resolution)
81         ys = np.arange(-extent, extent, resolution)
82         X, Y = np.meshgrid(xs, ys)
83
84         flat_X = X.ravel()
85         flat_Y = Y.ravel()
86
87         exact_scores = self.board.get_score(flat_X, flat_Y).reshape(X.shape)
88
89         # Apply Gaussian Filter to simulate inaccuracy
90         sigma_pixels = player.sigma / resolution
91         ev_map = gaussian_filter(exact_scores.astype(float), sigma=sigma_pixels)
92
93         plt.figure(figsize=(10, 8))
94         plt.title(f"Expected Payoff Heatmap\nPlayer: {player.name}")
95         im = plt.imshow(ev_map, extent=[-extent, extent, -extent, extent], origin=
'lower', cmap='inferno')
96         plt.colorbar(im, label='Expected Score')
97
98         # Draw board rings
99         circles = [
100             self.board.R_BULLSEYE, self.board.R_OUTER_BULL,
101             self.board.R_TREBLE_IN, self.board.R_TREBLE_OUT,
102             self.board.R_DOUBLE_IN, self.board.R_DOUBLE_OUT
103         ]
104         ax = plt.gca()
105         for r in circles:
106             c = patches.Circle((0, 0), r, fill=False, color='white', alpha=0.3)
107             ax.add_patch(c)
108
109         max_idx = np.unravel_index(np.argmax(ev_map), ev_map.shape)
110         best_y = ys[max_idx[0]]
111         best_x = xs[max_idx[1]]
112
113         plt.plot(best_x, best_y, 'cx', markersize=10, markeredgewidth=2, label='
Optimal Aim')
114         plt.legend()
115         plt.show()

```

```

116
117 def get_bullseye_probability(sigma, r_bullseye=6.35):
118     if sigma <= 0: return 1.0
119     return 1 - np.exp(-(r_bullseye**2) / (2 * sigma**2))
120
121 def integrate_gaussian_over_polar_sector(sigma, r_min, r_max, theta_center_deg,
122     width_deg=18):
123     # Define the aim point at center of sector
124     r_aim = (r_min + r_max) / 2.0
125     theta_aim_rad = np.radians(theta_center_deg)
126     aim_x = r_aim * np.cos(theta_aim_rad)
127     aim_y = r_aim * np.sin(theta_aim_rad)
128
129     # Numerical integration setup
130     r_steps = 50
131     theta_steps = 50
132
133     rs = np.linspace(r_min, r_max, r_steps)
134     thetas = np.linspace(np.radians(theta_center_deg - width_deg/2),
135         np.radians(theta_center_deg + width_deg/2), theta_steps)
136
137     R, T = np.meshgrid(rs, thetas)
138     X = R * np.cos(T)
139     Y = R * np.sin(T)
140
141     rv = multivariate_normal([aim_x, aim_y], [[sigma**2, 0], [0, sigma**2]])
142     pos = np.dstack((X, Y))
143     pdf_values = rv.pdf(pos)
144     dr = (r_max - r_min) / (r_steps - 1)
145     dtheta = np.radians(width_deg) / (theta_steps - 1)
146
147     return np.sum(pdf_values * R * dr * dtheta)
148
149 def get_hit_probability(sigma, target_type="T20"):
150     board = Dartboard()
151     type_code = target_type[0]
152
153     if type_code == 'D':
154         r_min, r_max = board.R_DOUBLE_IN, board.R_DOUBLE_OUT
155     elif type_code == 'T':
156         r_min, r_max = board.R_TREBLE_IN, board.R_TREBLE_OUT
157     elif type_code == 'S':
158         r_min, r_max = board.R_TREBLE_OUT, board.R_DOUBLE_IN
159     else:
160         raise ValueError("Unknown target type")
161
162     return integrate_gaussian_over_polar_sector(sigma, r_min, r_max, 0)
163
164 if __name__ == "__main__":
165     board = Dartboard()
166     sim = Simulation(board)
167
168     sigma_pro = 10
169     sigma_amateur = 40
170
171     # Parameters (Pro Player)
172     p_treble = get_hit_probability(sigma_pro, "T")
173     p_double = get_hit_probability(sigma_pro, "D")
174     p_single = get_hit_probability(sigma_pro, "S")
175     p_bull = get_bullseye_probability(sigma_pro)
176     p_next = p_double * p_single
177     p_err = 0.05 # Small chance to recover if Bull is missed

```

```

177
178     print(f"Amateur aiming at bullseye probability: {get_bullseye_probability(
179         sigma_amateur)})")
180
181     print(f"--- Pro Player (Sigma={sigma_pro}mm) ---")
182     print(f"Prob of hitting Treble: {p_treble:.2%}")
183     print(f"Prob of hitting Double: {p_double:.2%}")
184     print(f"Prob of hitting Single: {p_single:.2%}")
185     print(f"Prob of hitting Bullseye: {get_bullseye_probability(sigma_pro):.2%}")
186
187     pro_player = Player("Pro Player", skill_sigma=sigma_pro)
188     avg_player = Player("Amateur", skill_sigma=sigma_amateur)
189
190     sim.visualize_heatmap(pro_player)
191     sim.visualize_heatmap(avg_player)
192
193     # --- Strategy Plotting ---
194     q_opp = np.linspace(0, 1, 100)
195
196     # Calculate Win Probabilities
197     P_safe = (1 - q_opp) * p_next
198     P_agg = p_bull + (1 - p_bull) * (1 - q_opp) * p_err
199
200     plt.figure(figsize=(8, 5))
201     plt.plot(q_opp, P_safe, label=r'Strategy Safe ( $\mu_{\text{Safe}}$ )', color='blue',
202             linewidth=2)
203     plt.plot(q_opp, P_agg, label=r'Strategy Aggressive ( $\mu_{\text{Agg}}$ )', color='red',
204             linewidth=2, linestyle='--')
205
206     # Find intersection
207     idx = np.argwhere(np.diff(np.sign(P_agg - P_safe))).flatten()
208     if len(idx) > 0:
209         intersect_x = q_opp[idx[0]]
210         intersect_y = P_agg[idx[0]]
211         plt.plot(intersect_x, intersect_y, 'ko')
212         plt.annotate(f'Critical Point\n$q \approx {intersect_x:.2f}$',
213                    xy=(intersect_x, intersect_y),
214                    xytext=(intersect_x + 0.1, intersect_y + 0.15))
215
216     plt.title('Strategic Indifference: Aggression vs. Safety')
217     plt.xlabel(r'Opponent's Threat Level ( $q_{\text{opp}}$ )')
218     plt.ylabel(r'Player A's Probability of Winning ( $P_{\text{win}}$ )')
219     plt.legend()
220     plt.grid(True, linestyle=':', alpha=0.6)
221     plt.xlim(0, 1)
222     plt.ylim(0, 0.7)
223
224     plt.savefig('zero_sum_plot.png', dpi=300)
225     plt.show()

```

Listing 1: Dartboard Simulation and Strategic Analysis